DP-2026-001-02-capability-typed-actuation

Actuation as a typed capability, such that an ungranted actuation cannot be expressed

> **Defensive publication.** This document is published solely to place its subject matter into the public domain of technical knowledge as prior art. **No patent protection is sought by the author for any subject matter described here.**
>
> **Author / declarant:** Jun Kawasaki (`root@junkawasaki.com`) **Disclosure set:** DP-2026-001, part 2 of 10 · first published 2026-08-19 (UTC) **Enabling reference implementation:** public source code under `https://github.com/kotoba-lang/`, licensed Apache-2.0, fixed by commit and tree SHA-1 in the evidence bundle at `https://github.com/com-junkawasaki/prior-art` — that bundle carries RFC 3161 time-stamp tokens and OpenTimestamps attestations over the fixing manifest.

Note on the "Variants and generalizations" section below

The variants are stated deliberately and at length. They are disclosed so that a later filing directed to an obvious variation of the mechanism is met by an express **written** disclosure of that variation, and not merely by an argument that the variation would have been obvious. Where the text says "any", the word is intended in its ordinary broad sense and the enumeration following it is illustrative, not exhaustive.

Working source code implementing the mechanism is publicly available at the commits fixed in the evidence bundle. A person of ordinary skill in robotics software can read, build and run that code; this document is therefore an enabling disclosure and not a mere statement of a result.

Problem

Section 1 places a check between the actor and the hardware. That check can be bypassed if the actor's language gives it *ambient authority* — the ability to reach a device, socket, file, or driver directly, without asking. Every practical safety mechanism built on filtering an output stream can be defeated by a component that writes to the device by another route.

Mechanism

The actor's program is written in a language, or a restricted subset of a language, in which the ability to cause an externally observable effect **is a value that must be received**, not an operation that is always available.

Specifically:

- The program declares the set of effects it may perform. The declaration is part of the compiled artifact.

- The compiler **infers** the effect set from the program body and **refuses to compile** if the program performs an effect it did not declare, and equally if it declares an effect it does not perform.
- At load time the host supplies a set of **grants**. If the grants do not match the artifact's required effect set exactly, **instantiation fails** — the program never begins executing, rather than failing at the moment it first attempts the effect.
- The program has no general-purpose escape: no arbitrary module loading, no `eval`, no reflection, no foreign-function interface, no ambient global mutable state, no direct access to devices, sockets, filesystems, or credentials. There is exactly one way for an effect to occur, and it is through a granted capability.
- Each effect is additionally bounded by quantitative limits enforced by the host: an execution-step or "fuel" budget, a memory ceiling, a queue depth, a message-size ceiling. Exhaustion is a defined, reported outcome, not undefined behaviour.
- Errors are returned as values (a result type carrying either the success value or a typed error), not thrown, so that no control-flow path silently skips a check.
- Policy is **deny-by-default**: absence of a grant is refusal, never permission.
- Capabilities are **attenuable on delegation**: a component may pass on a strictly narrower capability (narrower resource, shorter deadline, smaller budget) but never a broader one.

Applied to robotics, the effects are precisely the actuation and sensing surfaces: set joint effort, publish a velocity command, energize a tool, read a sensor. A policy that was granted "read the lidar" and "publish a velocity command bounded to 0.4 m/s" **cannot express** energizing a gripper, because the operation is not merely forbidden — it is not reachable from the program text.

The capability is identified by a **name in a scoped semantic model** (kind, resource, holder) rather than by a numeric wire index; numeric indices, if any, are an internal transport detail of the host and do not appear in the program's source.

Variants and generalizations

- The restricted language may be: a purpose-designed language; a subset of an existing language enforced by a verifier; a bytecode with a typed import table; WebAssembly with an explicitly declared import set; a sandboxed interpreter; a proof-carrying binary.
- The effect set may be: declared and checked; wholly inferred; inferred with declaration required only at public API and package boundaries; declared as a maximum ceiling with the inferred set required to be a subset.
- The grant may be: a plain token; a signed certificate; an object capability reference; a UCAN, CACAO, macaroon, biscuit, or comparable attenuable credential; a hardware-backed key; a lease with an expiry; a quorum-issued certificate. The binding of grant to artifact may be by exact set equality, by subset, or by a stated compatibility relation, provided that absence of a grant is refusal.
- The failure-closed point may be: compile time; link time; instantiation time; first use. Instantiation-time refusal is preferred because it removes the partially executed state, but each is a variant.
- Quantitative bounds may be: instruction counts; wall-clock deadlines; energy budgets; actuation-magnitude ceilings (velocity, force, torque, jerk, temperature); count of actuations; distance travelled; volume of workspace swept.

- The same arrangement covers non-robotic effects: network access, filesystem access, payment, message publication, model inference, code deployment.
- The host may be: a server process; a browser; an embedded runtime; a real-time executive; a microkernel; bare metal. The mechanism does not depend on an operating system being present.
- The capability namespace may be: flat names; hierarchical paths; URIs; content-addressed identifiers of the capability's own semantic definition.

Reference implementation

`kotoba-lang/kotoba-lang` (`lang/capability-semantics.edn`, `lang/surface-status.edn`), `kotoba-lang/amu` (capability kits, effect inference, admission), `kotoba-lang/kototama` (host linking), `kotoba-lang/capability-*` (the individual capability contracts, including `capability-kami-engine`, `capability-motion-read`, `capability-topic-publish`, `capability-topic-subscribe`, `capability-irq-subscribe`, `capability-mmio-map`, `capability-dma-map`).

中文摘要 / Chinese abstract

本文件为**防御性公开**（defensive publication）。作者自愿公开以下技术方案，使其成为**现有技术**（专利法第二十二条所称"为公众所知的技术"），并声明**不为其中任何技术方案申请专利**。参考实现的源代码自 2026 年 8 月 19 日起在互联网上公开获取，采用 Apache-2.0 许可，仓库地址与提交哈希见 `https://github.com/com-junkawasaki/prior-art`。

将执行机构操作表示为带类型的能力（capability），使未被授予的操作无法被表达：程序所声明的副作用集合由编译器从程序体**推断**，声明与推断不一致即拒绝编译；装载时若宿主提供的授权与制品所需能力集合不一致，则**实例化失败**，程序根本不会开始执行。语言中不存在任意模块装载、`eval`、反射、外部函数接口、环境全局可变状态，也不存在对设备、套接字、文件系统或凭据的直接访问；错误以值（result 类型）返回而非抛出；策略为**默认拒绝**；宿主强制执行燃料（步数）、内存、队列深度与消息大小上限。因此一个仅被授予"读取激光雷达"和"以不超过 0.4 m/s 发布速度指令"的策略，**无法表达**给夹持器通电这一操作。

上文"变型与推广"一节系**有意穷举列举**，以使针对本机制之显而易见变化的在后申请面对的是明确的书面公开，而非仅仅是"显而易见"的主张。

Statement of dedication